reduce the startup time. After the startup phase, the application will use much lower CPU resources.

A common strategy is to define the CPU request (`resources.requests.cpu`) with the amount the application will use under normal conditions, so that it's always guaranteed to have the resources required to operate correctly. Then, depending on the system, you may decide to specify a higher CPU limit or omit it entirely (`resources.limits.cpu`) to optimize performance at startup so that the application can use as much CPU as available on the node at that moment.

CPU is a *compressible resource*, meaning that a container can consume as much of it as is available. When it hits the limit (either because of `resources.limits.cpu` or because there's no more CPU available on the node), the operating system starts throttling the container process, which keeps running but with possibly lower performance. Since it's compressible, not specifying a CPU limit can be a valid option sometimes to gain a performance boost. Still, you'll probably want to consider the specific scenario and evaluate the consequences of such a decision.

Unlike CPU, *memory* is a *non-compressible resource*. If a container hits the limit (either because of `resources.limits.memory` or because there's no more memory available on the node), a JVM-based application will throw the dreadful `OutOfMemoryError`, and the operating system will terminate the container process with an `OOMKilled` (OutOfMemory killed) status. There is no throttling. Setting the correct memory value is, therefore, particularly important. There's no shortcut to inferring the proper configuration; you must monitor the application running under normal conditions. That's true for both CPU and memory.